Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет информационных технологий и программирования
Кафедра прикладной информатики

ЛАБОРАТОРНАЯ РАБОТА №3

Контейнеризация написанного приложения средствами Docker

Выполнил: студент группы К3341
Кижваткин Н.М.

Санкт-Петербург
2025

1. ЦЕЛЬ И ЗАДАЧИ

Цель работы: контейнеризация системы Callmentor с использованием Docker и Docker Compose для обеспечения воспроизводимого окружения разработки и развёртывания.

Задачи: написать Dockerfile для каждого сервиса приложения, составить docker-compose.yml для локального запуска всей системы, настроить healthcheck-проверки, тома для персистентности данных, сетевые зависимости между сервисами.

2. КОНТЕЙНЕРИЗАЦИЯ МИКРОСЕРВИСОВ

2.1. Gateway (FastAPI)

Gateway использует базовый образ Python. Dockerfile включает: копирование requirements.txt и установку зависимостей через pip, копирование исходного кода, запуск через uvicorn. Особенности: монтируется том с исходным кодом для hot-reload в режиме разработки, отдельный том /reports/ для хранения сформированных Excel-отчётов.

2.2. Frontend (React/Vite)

Frontend использует многоэтапную сборку (multi-stage build): Stage 1 (build) — Node.js образ, npm install, npm run build; Stage 2 (runtime) — Nginx образ, копирование dist/ из первого этапа. Это минимизирует размер финального образа, исключая devDependencies и инструменты сборки.

2.3. ML-сервисы (ml-service, ml-asr-worker, ml-llm-worker)

ML-сервисы используют базовые образы Python. Особенности контейнеризации: ml-asr-worker требует значительного объёма памяти для ML-моделей (PyTorch, T-one), общий том ./data/calls монтируется во все ML-сервисы для передачи аудиофайлов между ними, команда запуска в dev-режиме использует watchmedo для автоперезагрузки при изменении кода.

2.4. Nginx (Reverse Proxy)

Nginx Docker-образ включает кастомный конфигурационный файл default.conf. Nginx маршрутизирует запросы: /api/* → gateway:8000 (REST API), /* → frontend:3000 (React SPA), /audio-proxy/* → проксирование аудиофайлов с CRM-систем с whitelist хостов.

3. КОНФИГУРАЦИЯ DOCKER COMPOSE

3.1. Граф зависимостей сервисов

Зависимости сервисов определены через depends_on с condition проверками:

Таблица 1 — Зависимости сервисов

Сервис	Зависимости (condition)
nginx	gateway (service_started), frontend (service_started)
gateway	gateway-db (service_started), ml-service (service_healthy), ml-asr-worker (service_started), ml-llm-worker (service_started)
ml-service	rabbitmq (service_healthy), ml-redis (service_started), ml-asr-worker (service_started), ml-llm-worker (service_started)
ml-asr-worker	rabbitmq (service_healthy), ml-redis (service_started)
ml-llm-worker	rabbitmq (service_healthy), ml-redis (service_started)

3.2. Healthcheck-конфигурация

Таблица 2 — Healthcheck-конфигурация сервисов

Сервис	Команда проверки	Параметры
ml-service	curl -f http://localhost:8000/ml/healthcheck	interval: 120s, retries: 5
ml-llm-worker	curl -f http://localhost:8000/llm/healthcheck	interval: 120s, retries: 5
ml-asr-worker	curl -f http://localhost:8000/asr/healthcheck	interval: 120s, retries: 5
rabbitmq	rabbitmq-diagnostics -q check_running	interval: 10s, retries: 5
gateway-db	pg_isready -U ${POSTGRES_USER}	interval: 10s, retries: 5 (prod)
frontend (prod)	curl -f http://localhost/	interval: 30s, retries: 3

3.3. Управление томами

Таблица 3 — Именованные тома Docker

Том	Назначение
gateway_postgres_data (gateway-volume-db)	Данные PostgreSQL (бизнес-данные)
rabbitmq_data	Персистентные очереди RabbitMQ
grafana-data	Конфигурация и данные Grafana
loki-data	Хранилище логов Loki
prometheus-data	Метрики Prometheus
./data/calls (bind mount)	Аудиофайлы звонков (общий для ML-сервисов)
./data/reports (bind mount)	Сформированные Excel-отчёты

4. КОНФИГУРАЦИЯ ОКРУЖЕНИЯ

Конфигурация сервисов вынесена в .env-файлы. В репозитории хранятся example.env шаблоны с заглушками для всех переменных. Переменные среды включают: POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB, POSTGRES_HOST, POSTGRES_PORT, SECRET_KEY, ALGORITHM, RABBITMQ_USER, RABBITMQ_PASSWORD, PROCESS_AUDIO_URL, TASK_STATUS_URL, VASTAI_IP_POST. Каждый сервис подключает свой .env через env_file.

5. СИСТЕМА СТЕКА OBSERVABILITY В DOCKER COMPOSE

Стек наблюдаемости реализован как набор сервисов в docker-compose.yml:

- Grafana монтирует директорию observability/grafana-provisioning с преднастроенными дашбордами и источниками данных (provisioning).
- Prometheus использует конфигурационный файл observability/prometheus/prometheus.yml.
- Promtail монтирует /var/lib/docker/containers и /var/run/docker.sock для доступа к логам контейнеров.
- Loki использует конфигурацию observability/loki-config.yml.

6. ЗАПУСК СИСТЕМЫ

Для запуска локальной среды разработки используется Makefile с набором команд: make up — запуск всех сервисов (docker compose up -d), make down — остановка, make logs — просмотр логов. Последовательность запуска автоматически управляется через depends_on. После запуска система доступна по адресу http://localhost:1080 (Nginx proxy).

7. ВЫВОДЫ

Произведена полная контейнеризация системы Callmentor. Docker Compose файл описывает 16 сервисов с настроенными зависимостями, healthcheck-проверками, томами для персистентности данных и конфигурацией через переменные окружения. Использование именованных томов обеспечивает сохранность данных при перезапуске контейнеров. Многоэтапная сборка frontend минимизирует размер образа.